Base de datos key-value replicada

PCP · Segundo cuatrimestre 2026 · Diseño según consigna V5

1. Estado, interfaz y TTL

La implementación corre en una única VM BEAM. Se asume que los procesos no fallan y que stop/1 se invoca una vez, al final, con todas las réplicas. simple_kv exporta start_link/2, stop/1, put/3, put/4, delete/2 y get/2.

start_link(Count, Alfa) valida Count entero positivo y Alfa entero ≥3 en el invocante, crea Count procesos con spawn_link y comunica sus pares y capacidad. Los TTL admitidos son enteros no negativos e infinity; cualquier otro falla con function_clause antes de enviar mensajes. put/3 usa infinity. Claves y valores admiten cualquier término Erlang.

Cada réplica mantiene exclusivamente en argumentos de su ciclo:

• Peers: identificadores de las otras réplicas; Alfa: capacidad máxima.

• Map: Key => {Version, {value, Val, Vence}}. Es el único almacenamiento de valores de la réplica; contiene como máximo Alfa entradas.

• Metadatos: Key => {Version, valor | borrado | desalojado}. Conserva versión incluso sin valor: un desalojo es local, un borrado invalida el dato.

• Locales: claves y versiones de las últimas Q claves distintas escritas localmente, con Q = ceil(3·Alfa/4). No contiene valores y tiene como máximo Q entradas.

• Pendientes: ConsultaRef => {Client, Ref, Key, Faltan}. Registra lecturas remotas; no almacena resultados ni valores.

Los metadatos pueden crecer con la cantidad de claves conocidas. No se recolectan lápidas: quitarlas sin demostrar que ya no quedan mensajes antiguos permitiría resurrecciones. La cota Alfa corresponde a valores, no a claves o versiones de metadatos.

El vencimiento se calcula como monotonic_time(millisecond) + TTL una única vez al escribir; infinity se conserva. Todas las réplicas comparten reloj monotónico en la VM. Replicar u ofrecer conserva el instante absoluto: recibir una copia no renueva su vida.

Un valor es vigente cuando ahora < Vence. La API vuelve a comprobar TTL al finalizar una lectura, incluso si el candidato llegó antes y venció mientras faltaban respuestas. TTL cero reemplaza lógicamente una escritura anterior por información vencida, sin desalojar otra clave. Los vencidos se retiran también al reservar espacio y recibir ofertas, conservando versión como borrado.

No se usan behaviours OTP, ETS, DETS, Mnesia, persistent_term ni diccionario del proceso. Los módulos auxiliares operan sobre los mapas recibidos como argumentos.

2. Protocolo de comunicación

Abreviaturas: C = cliente, R = réplica, P = par, I = invocante de inicio/cierre. E = {Version, {value, Val, Vence}} o {Version, borrado}. Resultado = error o {ok, Entry}, incluyendo marcas borrado/desalojado sin valor.

Mensaje	Emisor → receptor	Efecto
{peers, Peers, Alfa}	I → R	Inicializa pares y capacidad; empieza el ciclo.
{put, C, Ref, Key, Val, TTL}	C → R	Genera versión, reserva espacio y guarda; confirma y replica.
{delete, C, Ref, Key}	C → R	Genera lápida, elimina valor, confirma y replica.
{Ref, ok}	R → C	Termina put/delete sin esperar confirmaciones de pares.
{get, C, Ref, Key}	C → R	Responde con lo local o inicia consulta a pares.
{Ref, candidato, Resultado}	R → C	Entrega un candidato de lectura; el cliente reduce por versión.
{Ref, fin}	R → C	No quedan respuestas: el cliente comprueba TTL y retorna.
{replica, Key, E}	R → todos sus P	Aplica versión aceptable; controla capacidad si es valor. No confirma ni vuelve a replicar.
{ofrecer, Key, E}	R → todos sus P	Ofrece un valor antes de descartarlo. Se acepta solo vigente, sin versión más nueva y con espacio o reemplazo. No se reenvía.
{consultar, R, ConsultaRef, Key}	R → P	P contesta solo con su estado local; no inicia otra consulta.
{respuesta_consulta, ConsultaRef, P, Resultado}	P → R	Si P falta en esa consulta, reenvía candidato a C y elimina P de Faltan.
stop	I → R	R termina normalmente. No manda confirmación explícita.
{'DOWN', Monitor, process, R, Motivo}	VM → I	Confirma terminación de R al monitor creado por stop/1.

Cada operación usa make_ref(). Las consultas remotas tienen una referencia distinta de la del cliente. Una respuesta solo se acepta si la consulta existe y su emisor está en Faltan; así se descartan referencias desconocidas, duplicados y pares ajenos. Al terminar se elimina Pendientes[ConsultaRef].

El ciclo descarta mensajes inesperados; la inicialización también descarta mensajes que no sean su configuración. No hay reintentos ni confirmaciones para replicaciones/ofertas. La comunicación asincrónica se implementa con !; las funciones públicas esperan su propia referencia. stop/1 monitorea, envía stop y espera DOWN sin imponer un timeout que contradiga la espera de terminación.

3. Versiones, conflictos y borrados

Cada put/delete crea una versión con erlang:unique_integer([monotonic]). En la VM es única y creciente. La réplica de origen la genera una sola vez y todos los mensajes conservan esa versión. No se usa el orden de llegada como criterio de conflicto.

Una versión mayor reemplaza la información anterior; una menor se ignora. Para versiones diferentes, elegir el máximo es conmutativo, asociativo e idempotente: reordenar o duplicar mensajes no cambia el ganador. No hay empate entre dos escrituras originales distintas. Las versiones iguales provienen de copias de una misma operación.

Un delete produce {Version, borrado}. Aunque se elimine el valor de Map, Metadatos conserva la lápida. Un put atrasado con versión menor pierde contra esa lápida. Una nueva escritura posterior sí gana, como exige la consigna. Recibir un delete viejo tampoco elimina un valor nuevo.

Desalojar conserva {Version, desalojado}. Esta marca no significa que el dato dejó de existir: habilita la consulta a pares y la recuperación de una copia de igual versión. Para igual versión, aplicar_actualizacion/4 solo repone un valor cuando el estado era desalojado. elegir_mayor/2 prefiere el valor a la marca de desalojo de esa misma versión. Una lápida de borrado no se revierte mediante esa excepción.

La limpieza de un valor vencido conserva su versión como borrado. No crea una escritura nueva. Una copia de igual versión tiene el mismo vencimiento absoluto y tampoco es visible después de ese instante, aun si aún conserva físicamente el valor.

Convergencia y capacidad

Una vez detenidas las escrituras, todas sus replicaciones se procesan y las ofertas también terminan. La versión máxima de cada clave llega a todos los nodos, incluso si alguno debe descartarla por capacidad: conserva la versión con estado desalojado. No reaparecen versiones anteriores ni se inventan versiones al ofrecer.

Con capacidad finita, igualdad lógica de versiones y resolución de conflictos no exige que todos guarden físicamente los mismos valores: las copias se pueden desalojar de manera distinta y los metadatos distinguen esa ausencia local de un borrado. Las lecturas consultan los pares para encontrar una copia válida. Si todos pierden una clave por saturación, no existe un valor para devolver; la V5 admite esa pérdida en el peor caso.

Esta interpretación compatibiliza consistencia eventual con desalojo local y protección del 75 % local: para una clave conservada, las copias vigentes resuelven el mismo ganador; una ausencia por desalojo nunca se usa como una escritura que invalide otras copias.

4. Capacidad, protección local y ofertas

Se adopta Q = ceil(3·Alfa/4), redondeando hacia arriba para conservar al menos el 75 %. La recencia se mide por versión de escritura, sobre claves distintas escritas en ese nodo; una sobrescritura renueva la posición de su clave. Los borrados, vencimientos o reemplazos por una versión más nueva invalidan la obligación de devolver el valor anterior.

Locales guarda las últimas Q claves distintas y sus versiones locales. Al registrar una escritura se actualiza su clave; si hay Q+1, se elimina la de menor versión. Mantener allí una referencia a una versión ya borrada o reemplazada no conserva su valor ni protege versiones distintas de esa misma clave.

Para guardar un valor vigente se limpian vencidos. Si su clave ya está o queda espacio, se aplica la actualización. En caso contrario se elige la versión menor entre las entradas no protegidas. Una entrada está protegida solo si su clave y versión coinciden exactamente con Locales.

Una escritura local protegida entra desalojando una elegible. Para una entrada remota no protegida se compara también su versión: si es más vieja que las elegibles, o todos los residentes están protegidos, se descarta la propia entrada recibida. Antes se ofrece y se conserva su versión como desalojado. Una recepción remota no puede desplazar los valores locales protegidos de otras claves.

Argumento de la cota y de la protección

El estado inicial tiene cero valores. Cada transición que inserta reemplaza una clave existente, usa una vacante o elimina una entrada antes de insertar. Nunca se construye un mapa con Alfa+1 entradas. Deletes y limpieza solo reducen tamaño. Una oferta solo entra con vacante o reemplazo. Por inducción, todos los estados de almacenamiento respetan la cota Alfa.

Una escritura local nueva está entre las Q protegidas. Si el mapa está lleno y su clave no está, como máximo Q-1 residentes están protegidos: la clave entrante ocupa una posición de ese cupo. Como Q ≤ Alfa, hay al menos un residente elegible, incluso para Alfa=3. Por eso la nueva escritura entra sin quitar otra protegida. Una recepción remota de otra clave tampoco las desaloja. Los valores de esas últimas Q escrituras distintas permanecen disponibles mientras sigan vigentes y no sean reemplazados/borrados.

Oportunidad de conservación y terminación

Antes de descartar un vigente se envía ofrecer a todos los pares. Un receptor acepta únicamente si está vigente, la versión es aceptable y hay lugar o reemplaza una clave existente. Puede limpiar vencidos, pero nunca desaloja para aceptar ni emite otros mensajes. Por tanto una oferta no produce nuevas ofertas.

Un put o una replicación puede causar como máximo un descarte y una oferta por par. Recibir una replicación no la vuelve a propagar. Para un número finito de escrituras hay un número finito de estos mensajes. La pérdida es posible si ningún receptor puede conservar el valor, y no genera una cadena infinita ni espera circular.

5. Lecturas, progreso y validación

Si el estado local es valor o borrado, la réplica envía candidato y fin al cliente. Sin pares también responde con lo local. Si es desconocido o desalojado, envía el candidato local inicial, registra Pendientes y consulta una sola vez a cada par.

Cada par contesta inmediatamente con lo que conoce, sin consultar a otros. La réplica solicitante reenvía cada respuesta válida al cliente y actualiza Faltan. Al llegar la última, vuelve a leer su estado local (pudo cambiar mientras esperaba), envía ese candidato y después fin. Todos esos mensajes hacia el cliente salen del mismo proceso; el orden de Erlang garantiza que fin no adelante candidatos anteriores.

La reducción por versión ocurre en la función pública get/2, en el proceso cliente, no en la réplica. El cliente mantiene el mejor resultado hasta fin y comprueba su vencimiento justo antes de retornar {ok, Val} o not_found. Pendientes solo conserva claves, referencias e identificadores: los resultados remotos no constituyen una caché ni almacenamiento secundario de valores del nodo. Los mensajes en tránsito son parte del protocolo; no se guardan en otra estructura de la réplica.

No hay lectura con garantía de instantánea o linealizabilidad entre escrituras concurrentes; se usa la mejor información recibida y una comprobación local final. Cuando cesan las escrituras y llega la propagación, rige el criterio eventual descrito anteriormente.

Ausencia de deadlock

La réplica que consulta vuelve al ciclo normal: puede atender puts, deletes, ofertas y consultas mientras faltan pares. Responder una consulta no depende de una respuesta de otro proceso. Por eso no hay arista de espera bloqueante entre réplicas, aun cuando se consulten mutuamente. Con procesos que no fallan y ejecución eventual de los mensajes, todos los pares responden; el cliente recibe fin. No se promete latencia acotada ni recuperación de procesos fallidos, que quedan fuera de los supuestos.

Validación reproducible

rebar3 compile y rebar3 eunit compilan con warn_all y warnings_as_errors. También se puede compilar directamente con erlc +warn_all -Werror. Los tests cubren API y validaciones, enlaces, términos compuestos, TTL y reemplazos, propagación, conflictos desordenados/concurrentes, borrados sin resurrección, capacidad receptora, selector, ofertas y lecturas remotas.

Las regresiones V5 cubren el cupo local para Alfa de 3 a 16, tráfico remoto y escritores locales concurrentes, TTL cero con capacidad llena, cierre demorado más de un segundo, descarte durante inicialización, respuestas duplicadas/ajenas y vencimiento de un candidato mientras falta otro par. Se combinan pruebas por API pública con pruebas internas del protocolo para forzar órdenes concretos; las internas complementan, no sustituyen, el comportamiento observable.

Las pruebas finitas no demuestran todos los intercalados. La cota, protección local, elección de versiones y terminación se justifican por las invariantes anteriores. La verificación administrativa y el push al repositorio asignado por la cátedra corresponden a los integrantes; el repositorio privado de trabajo no constituye por sí solo la entrega.